

Understanding Parameters and Hyperparameters

Thomas E. Allen

DSC 305 Applied Machine Learning

In machine learning, the terms *parameter* and *hyperparameter* have distinct meanings.

A **parameter** is a variable that is learned by the model during the training process. These values define the relationship between the input features and the target variable, directly affecting the model's predictions.

A **hyperparameter** is a user-defined configuration value that impacts the learning process but is not learned by the model itself. Hyperparameters are set by the user before training and are used to control various aspects of the model, such as its complexity or the trade-off between bias and variance.

Example: Logistic Regression

Let's discuss these using logistic regression as an example:

Parameters

1. Weights ($w_1, w_2, \dots, w_D$): These are the coefficients for each input feature and are learned by the model during training.
2. Bias (w_0): This is the intercept term in the decision boundary equation and is also learned by the model during training.

Hyperparameters

1. Regularization constant (C): This is a hyperparameter that controls the strength of the regularization term added to the loss function. Regularization is used to prevent overfitting, and the value of C determines the balance between fitting the training data and keeping the model simple for better generalization.
2. Regularization type (L1 or L2 penalty): This is a hyperparameter that determines the type of regularization applied to the loss function. L1 regularization adds the absolute values of the weights, while L2 regularization adds the squared values of the weights. The choice between L1 and L2 regularization can affect the model's sparsity and generalization performance.

Not Everything Is a Parameter or Hyperparameter

It's important to note that some aspects of the training process are neither parameters nor hyperparameters. For example:

1. Probability estimates: These are the predicted probabilities output by the logistic regression model. They are not considered parameters or hyperparameters because they are derived from the learned parameters and do not directly impact the learning process.
2. Max number of iterations (`max_iter`): This is not a traditional hyperparameter because it is not typically tuned during a grid search or random search. Instead, `max_iter` serves as a stopping criterion for the optimization algorithm to prevent it from running indefinitely. If convergence is not achieved, the user can increase the value or take other steps to ensure convergence.
3. Choice of random state seed (`random_state`): This is neither a parameter nor a hyperparameter. It is a seed value used to initialize the random number generator that affects the training process, such as random weight initialization and data shuffling. Setting a specific `random_state` ensures that the results of the training process are reproducible. Note that you would *never* tune the

`random_state` parameter during a grid search or random search because that would invalidate the results when you evaluate your model.

It's worth mentioning that the term "parameter" has a different meaning in programming languages. In the context of programming, parameters are input values passed to functions or methods during a function call. In this sense, aspects like `random_state` would be considered Python parameters to a method, but they are not machine learning parameters.

K-Nearest Neighbors (KNN)

Parameters

1. No explicit parameters are learned by the model during training. The model stores the entire training dataset and uses it during the prediction phase to find the nearest neighbors. Since it has no parameters, KNN is considered a **non-parametric** method.

Hyperparameters

1. Number of neighbors (k): This is a hyperparameter that determines how many nearest neighbors the model considers when making predictions. A larger value of k can lead to a smoother decision boundary, while a smaller value of k can lead to a more complex, flexible decision boundary.
2. Distance metric: This is a hyperparameter that specifies the method used to calculate the distance between data points. Common choices include Euclidean distance, Manhattan distance, and Minkowski distance. The choice of distance metric can affect the model's performance depending on the structure of the data.

Support Vector Machines (SVM)

Parameters

1. Support vectors: These are the data points that lie on or closest to the decision boundary, and they are learned by the model during training.
2. Lagrange multipliers: These are variables associated with each support vector and are also learned by the model during training. They are used to determine the optimal decision boundary. (Note: We didn't cover this in class, so you don't need to memorize it for the exam!)

Hyperparameters

1. Kernel function (e.g., linear, polynomial, or radial basis function): The choice of kernel function determines the transformation applied to the input data to create a higher-dimensional space. Different kernel functions can lead to different decision boundaries and generalization performance.
2. Regularization parameter (C): This hyperparameter controls the trade-off between achieving a large margin and minimizing classification errors. A smaller value of C creates a wider margin but allows some misclassifications, while a larger value of C aims for a smaller margin with fewer or no misclassifications.
3. Kernel coefficient (gamma): This hyperparameter is specific to the radial basis function (RBF) kernel and controls the shape of the decision boundary. A small gamma value will produce a more flexible decision boundary, while a large gamma value will produce a more rigid decision boundary.
4. Degree of the polynomial kernel function (degree): This hyperparameter is specific to the polynomial kernel and determines the degree of the polynomial used for the transformation. A higher degree may lead to more complex decision boundaries but could also increase the risk of overfitting.

Hopefully, this helps you understand the distinction between parameters and hyperparameters in machine learning. If you have additional questions, please reach out! I'm glad to help!